Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is an efficient algorithm for computing the Discrete Fourier Transform (DFT) and its inverse. The Fourier Transform is fundamental in signal processing, as it transforms a signal from its time domain into its frequency domain. The FFT significantly reduces the computational complexity of performing a DFT from $O(n^2)$ to $O(n \log n)$, where n is the number of data points in the signal.

Introduction to FFT

The Discrete Fourier Transform (DFT) is used to analyze the frequency components of a discrete signal. It expresses a sequence of n complex numbers $\times 0, x1, \ldots, xn-1$ as a sum of sinusoids, breaking the signal down into its frequency components. Mathematically, the DFT of a sequence is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n / N}$$

where:

- x_n represents the input signal in the time domain,
- X_k is the transformed signal in the frequency domain,
- N is the total number of points,
- i is the imaginary unit, and
- k corresponds to the index of the frequency component.

FFT is a faster algorithm to compute this DFT, and it works by recursively breaking down a DFT of any composite size $n = N_1 \times N_2$ into smaller DFTs of sizes N_1 and N_2.

Applications in Signal Processing

FFT is widely used in digital signal processing (DSP) and has numerous practical applications:

- **Audio and Music Processing:** FFT helps analyze and process sound signals by breaking them down into frequency components. It is used for equalizers, sound compression algorithms (like MP3), and identifying musical notes in a signal.
- **Image Processing:** In image processing, FFT is used to process images by converting spatial data into frequency data. This allows for image filtering, noise reduction, and edge detection.

- **Communication Systems:** In modern communication, FFT is essential for converting signals into their frequency components for modulation and demodulation in systems such as Orthogonal Frequency Division Multiplexing (OFDM), used in Wi-Fi and LTE.
- **Seismic Data Analysis:** FFT is used to analyze earthquake data and other geophysical signals by transforming the data into the frequency domain to identify hidden patterns.
- **Radar and Sonar:** FFT is applied in radar and sonar systems to detect objects by analyzing the frequency shift of reflected signals.

Pattern Matching Algorithms

Naive String Matching Algorithm

The Naive String Matching Algorithm is the simplest string matching approach. It checks for the presence of a substring (pattern) in a larger string (text) by sliding the pattern one character at a time and comparing characters.

Steps:

- Slide the pattern from the start to the end of the text.
- At each position, compare the pattern with the substring of the text.
- If all characters match, return the position. If not, move to the next position.

Time Complexity:

Worst-case: $O((n - m + 1) * m)$ where n is the length of the text and m is the length of the pattern.

```
Example:
Text: "ABCABCD"
Pattern: "ABC"
Matches found at index: 0 and 4
```

Knuth-Morris-Pratt (KMP) Algorithm

The KMP Algorithm improves on the naive approach by reducing redundant comparisons. It preprocesses the pattern to determine the "longest proper prefix which is also a suffix" (LPS) array, which helps avoid unnecessary shifts during matching.

Steps:

- Preprocess the pattern to compute the LPS array.
- Use the LPS array to skip some comparisons while sliding the pattern over the text.

Key Idea:

If a mismatch occurs after matching k characters, the KMP algorithm uses the LPS array to determine where the next comparison should begin.

Time Complexity:

Both preprocessing and matching have linear time complexity: O(n+ m).

Example 1. `Text: "ABABABCA"`
`Pattern: "ABAB"`
LPS Array: $[0, 0, 1, 2]$
Małches found at index: 0 and 2

Boyer-Moore Algorithm

The Boyer-Moore Algorithm is one of the most efficient string matching algorithms in practice. It preprocesses the pattern and uses two heuristics to skip sections of the text: the Bad Character Heuristic and the Good Suffix Heuristic.

Steps:

- Use the Bad Character Heuristic to skip ahead when a mismatch occurs by aligning the last occurrence of the mismatched character in the pattern with the mismatched text character.
- Use the Good Suffix Heuristic when the last characters match, skipping sections based on matched suffixes.

Time Complexity:

Best-case: O(n/m)
Worst-case: O(n * m) (less common due to the heuristics).

Example 2. `Text: "HERE IS A SIMPLE EXAMPLE"`
`Pattern: "EXAMPLE"`
`Bad Charac␣ler Heuristic helps skip several unnecessary comparisons.`
`Match found at index: }1`